

Projet : la transformation de Tseitin.

1 Avant-propos

Les évaluations de TP, notamment ce projet, sont des examens universitaires comptant pour la délivrance du diplôme national de licence.

À ce titre, toute fraude ou plagiat entraîne la rédaction d'un procès verbal à l'attention de la section disciplinaire de l'université, qui peut infliger, en plus de la nullité de l'épreuve (note de 0), une sanction allant de l'avertissement à l'exclusion définitive de tout établissement public d'enseignement supérieur.

Ainsi, si vous utilisez un morceau de code que vous n'avez pas produit, vous devez indiquer très clairement sa source (site internet, camarade de promotion, *etc.*) : vous ne serez pas accusé de plagiat, mais ce morceau de code ne sera pas compté dans votre note finale.

2 Modalités de réalisation et de rendu

Le projet est à réaliser par groupes de **deux étudiants maximum**. Vous devrez rédiger un rapport de projet contenant notamment les choix de programmation ou les difficultés rencontrées et les solutions apportées.

Vous devrez rendre le projet (rapport et code source dans une archive) pour le Vendredi 24 mai au plus tard, en déposant celui-ci sur universitice dans le dépôt correspondant. Une soutenance de 15 minutes par binôme aura lieu à partir du lundi 26 mai afin de vous interroger sur votre réalisation : vous serez interrogés au tableau sur l'application de l'algorithme (détaillé dans la suite) sur une formule donnée, puis devrez expliquer les différentes étapes de cet algorithme dans votre code.

3 Introduction

La transformation de Tseitin, due au mathématicien Grigori Tseitin, est une méthode permettant de modifier une formule afin d'éviter une explosion combinatoire potentielle lors de la mise en forme clausale conjonctive.

Le but de ce projet est d'implanter cet algorithme en OCaml ainsi qu'une méthode permettant de tester empiriquement vos résultats à l'aide de formules construites (pseudo-)aléatoirement.

Dans la suite sont détaillés les différents algorithmes que vous devrez mettre en œuvre.

4 Transformation de Tseitin

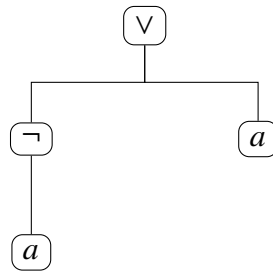
4.1 Idée de la méthode

La transformation de Tseitin est basée sur l'idée suivante : lors de la mise en forme clausale, des distributions entre les conjonctions et les disjonctions peuvent induire une explosion combinatoire de la taille de la forme produite ; au lieu de calculer cette distribution, on peut ajouter de nouveaux atomes représentant des sous-formules non atomiques de la formule de départ et inclure leurs définitions à l'aide de l'opérateur d'équivalence. Cet ajout permet de maintenir une taille linéaire lors de la mise en forme clausale.

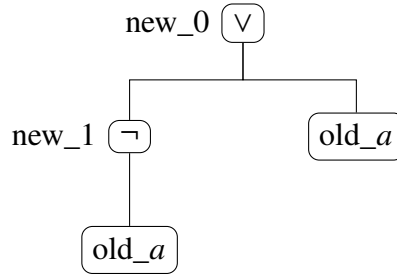
Cependant, la formule $T(F)$ produite n'est pas équivalente à la formule F de départ. En effet, les alphabets sont différents. Néanmoins, les deux formules sont **équisatisfaisables** : $T(F)$ est satisfaisable si et seulement si F l'est.

4.2 Exemples

Considérons la formule $F = \neg a \vee a$.



Chaque sous-formule est associée à un nouvel atome. Pour éviter toute ambiguïté, les anciens atomes sont alors préfixés par *old_* et les nouveaux par *new_*, utilisés pour *marquer* les sous-formules.



Ainsi, la formule de départ est transformée pour obtenir la formule $T(F)$ suivante :

$$\text{new_0} \quad (1)$$

$$\wedge ((\text{new_0} \leftrightarrow (\text{new_1} \vee \text{old_a})) \quad (2)$$

$$\wedge (\text{new_1} \leftrightarrow \neg \text{old_a})). \quad (3)$$

La formule produite peut-être interprétée ainsi :

- (3) indique que *new_1* représente $\neg \text{old_a}$,
- (2) indique que *new_0* représente $\text{new_1} \vee \text{old_a}$, c'est-à-dire $\neg \text{old_a} \vee \text{old_a}$ en combinant avec (3),
- (1) indique que *new_0* doit être vérifié, c'est-à-dire que $\text{new_1} \vee \text{old_a}$, ou d'une façon équivalente $\neg \text{old_a} \vee \text{old_a}$ (la formule de départ), doit être vérifié.

Les formules F et $T(F)$ ne sont pas équivalentes, n'ayant pas les mêmes alphabets. Cependant, on peut montrer que les deux propositions suivantes sont équivalentes :

- il existe une interprétation i telle que l'évaluation de F selon i soit vraie ;
- il existe une interprétation i' telle que l'évaluation de $T(F)$ selon i' soit vraie.

En effet, si i' est une interprétation telle que l'évaluation de $T(F)$ selon i' soit vraie, il suffit de poser $i(a) = i'(\text{old_a})$ pour que i soit une interprétation évaluant F comme vrai. Les deux interprétations i'_1 et i'_2 évaluant $T(F)$ comme vrai permettent de déduire les interprétations i_1 et i_2 suivantes, évaluant F comme vrai :

$$\begin{aligned} i'_1(\text{new_0}) &= 1, \\ i'_1(\text{new_1}) &= 0, \\ i'_1(\text{old_a}) &= 1, \end{aligned} \qquad i_1(a) = i'_1(\text{old_a}) = 1,$$

$$\begin{aligned} i'_2(\text{new_0}) &= 1, \\ i'_2(\text{new_1}) &= 1, \\ i'_2(\text{old_a}) &= 0, \end{aligned} \qquad i_2(a) = i'_2(\text{old_a}) = 0.$$

Réciproquement, si i est une interprétation évaluant F à vrai, il suffit de poser pour tous les atomes a de F

$$i'(\text{old_a}) = i(a),$$

puis de construire les éléments manquants à l'aide des équivalences de $T(F)$ ajoutées lors de la construction, en commençant par les atomes dont les indices sont les plus grands :

$$i'(\text{new_1}) = \neg i'(\text{old_a}), \quad (\text{d'après (3)})$$

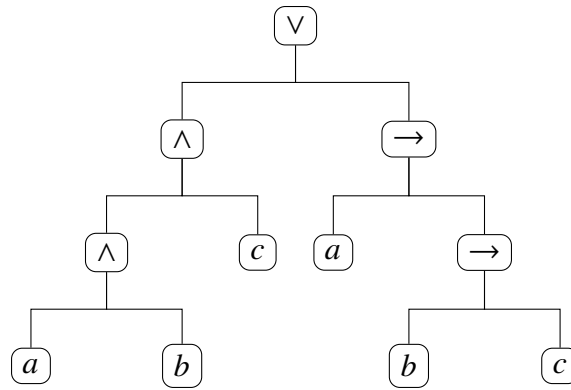
$$i'(\text{new_0}) = i'(\text{new_1}) \vee i'(\text{old_a}). \quad (\text{d'après (2)})$$

Les deux interprétations i_1 et i_2 évaluant F comme vrai permettent de déduire les interprétations i'_1 et i'_2 suivantes, évaluant $T(F)$ comme vrai :

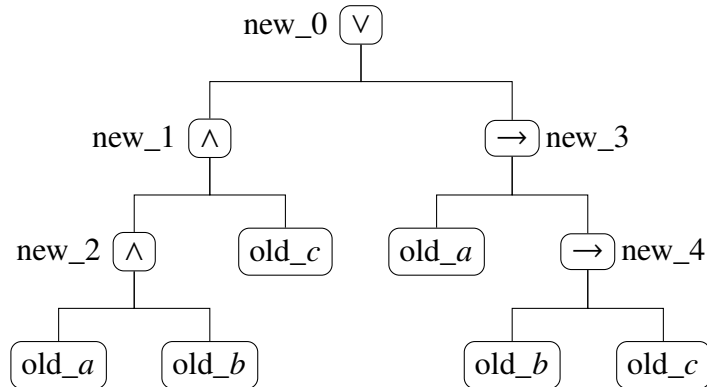
$$\begin{aligned} i_1(a) &= 1, & i'_1(\text{old_a}) &= i_1(a) = 1, \\ i'_1(\text{new_1}) &= \neg i'_1(\text{old_a}) = 0, \\ i'_1(\text{new_0}) &= i'_1(\text{new_1}) \vee i'_1(\text{old_a}) = 1, \end{aligned}$$

$$\begin{aligned} i_2(a) &= 0, & i'_2(\text{old_a}) &= i_2(a) = 0, \\ i'_2(\text{new_1}) &= \neg i'_2(\text{old_a}) = 1, \\ i'_2(\text{new_0}) &= i'_2(\text{new_1}) \vee i'_2(\text{old_a}) = 1. \end{aligned}$$

Considérons ensuite la formule $F_2 = a \wedge b \wedge c \vee (a \rightarrow b \rightarrow c)$.



Chaque sous-formule est associée à un nouvel atome. Pour éviter toute ambiguïté, les anciens atomes sont alors préfixés par *old_* et les nouveaux par *new_*, utilisés pour *marquer* les sous-formules.



Ainsi, la formule de départ est transformée pour obtenir la formule $T(F_2)$ suivante :

$$\begin{aligned}
 & \text{new_0} \\
 & \wedge ((\text{new_0} \leftrightarrow (\text{new_1} \vee \text{new_3})) \\
 & \wedge ((\text{new_1} \leftrightarrow (\text{new_2} \wedge \text{old_c})) \\
 & \wedge ((\text{new_2} \leftrightarrow (\text{old_a} \wedge \text{old_b})) \\
 & \wedge ((\text{new_3} \leftrightarrow (\text{old_a} \rightarrow \text{new_4})) \\
 & \wedge (\text{new_4} \leftrightarrow (\text{old_b} \rightarrow \text{old_c}))))))
 \end{aligned}$$

Comme lors de l'exemple précédent, les formules F_2 et $T(F_2)$ sont équisatisfaisables : on peut passer d'une interprétation i évaluant F comme vrai à une interprétation i' évaluant $T(F)$ comme vrai, et *vice versa*.

Passer d'une interprétation i' à une interprétation i se fait selon la même *restriction* de domaine que précédemment :

$$\begin{aligned}
 i'(\text{new_0}) &= 1 \\
 i'(\text{new_1}) &= 0 \\
 i'(\text{new_2}) &= 0 \\
 i'(\text{new_3}) &= 1 \\
 i'(\text{new_4}) &= 0 \\
 i'(\text{old_a}) &= 0 & i(a) = i'(\text{old_a}) = 0, \\
 i'(\text{old_b}) &= 1 & i(b) = i'(\text{old_b}) = 1, \\
 i'(\text{old_c}) &= 0 & i(c) = i'(\text{old_c}) = 0.
 \end{aligned}$$

Passer d'une interprétation i à une interprétation i' se fait selon la même *extension* de domaine que précédemment à l'aide des équivalences ajoutées lors de la construction de $T(F_2)$:

$$\begin{aligned}
 i(a) &= 1, & i'(\text{old_a}) &= i(a) = 1, \\
 i(b) &= 1, & i'(\text{old_b}) &= i(b) = 1, \\
 i(c) &= 1, & i'(\text{old_c}) &= i(c) = 1, \\
 & & i'(\text{new_4}) &= i'(\text{old_b}) \rightarrow i'(\text{old_c}) = 1, \\
 & & i'(\text{new_3}) &= i'(\text{old_a}) \rightarrow i'(\text{new_4}) = 1, \\
 & & i'(\text{new_2}) &= i'(\text{old_a}) \wedge i'(\text{old_b}) = 1, \\
 & & i'(\text{new_1}) &= i'(\text{new_2}) \wedge i'(\text{old_c}) = 1, \\
 & & i'(\text{new_0}) &= i'(\text{new_1}) \vee i'(\text{new_3}) = 1.
 \end{aligned}$$

4.3 Travail à réaliser

Dans le fichier `Tseitin.ml`, donner corps aux fonctions suivantes :

- `tseitin` calculant depuis une formule **ne contenant pas d'opérateur d'équivalence** sa transformée de Tseitin ;
- `restrict_inter` transformant une interprétation évaluant une transformée de Tseitin $T(F)$ comme vraie en une interprétation évaluant la formule F comme vraie ;
- `extension_inter` transformant une interprétation évaluant la formule F comme vraie en une interprétation évaluant sa transformée $T(F)$ comme vraie.

5

Génération aléatoire d'une formule pour un nombre d'opérateurs donné

La génération aléatoire d'une formule peut se réaliser en considérant la méthode récursive suivante.

Chercher à construire aléatoirement une formule avec n opérateurs et des atomes dans Σ , c'est :

- Si n vaut 0, renvoyer un atome de Σ choisi aléatoirement ;
- Si n vaut 1, choisir aléatoirement un des cas suivants :
 - un opérateur nullaire ($\perp$ ou $\top$) ;
 - un opérateur unaire (négation) et un atome de Σ ;
 - un opérateur binaire (disjonction, conjonction ou implication) et deux atomes de Σ ;
- Sinon, choisir aléatoirement un des cas suivants :
 - un opérateur unaire (négation) et une formule aléatoire avec $(n - 1)$ opérateurs ;
 - un opérateur binaire (disjonction, conjonction ou implication), un entier aléatoire k compris entre 0 et $(n - 1)$ inclus, et deux formules aléatoires avec respectivement k et $(n - k - 1)$ opérateurs.

La formule aléatoire générée ne doit pas contenir d'opérateur d'équivalence. Vous devrez utiliser le module `Random`, en vous aidant notamment de l'Annexe A pour implanter dans le fichier `RandomFormule.ml` la formule `random_form` prenant en paramètres un alphabet et le nombre d'opérateurs attendus.

6

Implantation : test empirique de validité des résultats : équisatisfaisabilité

Dans le fichier `CorrecTest.ml`, donner corps aux fonctions suivantes :

- `test_equisat_quine` prenant en paramètre une formule f (sans occurrence de l'opérateur d'équivalence) et qui réalise la séquence suivante :
 1. affiche f au format infixe,
 2. calcule la transformée f' de f et l'affiche au format infixe,
 3. détermine la satisfaisabilité de f selon l'algorithme de Quine et affiche le Booléen b correspondant,
 4. détermine la satisfaisabilité de f' selon l'algorithme de Quine et affiche le Booléen b' correspondant,
 5. détermine le Booléen $b = b'$ et l'affiche.
- `test_equisat_dp11` prenant en paramètre une formule f (sans occurrence de l'opérateur d'équivalence) et qui réalise la séquence suivante :
 1. affiche f au format infixe,
 2. calcule la transformée f' de f et l'affiche au format infixe,
 3. détermine la liste ts de tous les témoins de satisfaisabilité de f à l'aide de l'algorithme DPLL (fonction `dp11_all_sat`),
 4. détermine la liste ts' de tous les témoins de satisfaisabilité de f' à l'aide de l'algorithme DPLL (fonction `dp11_all_sat`),
 5. convertit chaque témoin de ts en une interprétation satisfaisant f , produisant une liste is de même longueur que ts ,
 6. convertit chaque témoin de ts' en une interprétation satisfaisant f' , produisant une liste is' de même longueur que ts' ,
 7. convertit chaque interprétation de is en une interprétation satisfaisant f' (en utilisant `extension_inter`), produisant la liste is_{ext} ,
 8. convertit chaque interprétation de is' en une interprétation satisfaisant f (en utilisant `restrict_inter`), produisant la liste is'_{res} ,
 9. calcule et affiche le Booléen correspondant au fait que toutes les interprétations de is_{ext} évaluent f' comme vrai,
 10. calcule et affiche le Booléen correspondant au fait que toutes les interprétations de is'_{res} évaluent f comme vrai.
- `test_equisat_quine'` et `test_equisat_dp11'` prenant en paramètres un alphabet et un nombre d'opérateurs, calculant une formule aléatoire en utilisant ces deux paramètres, puis appelant `test_equisat_quine` et `test_equisat_dp11` sur cette formule aléatoire.

Voici un exemple de sortie lors de l'exécution de ces deux dernières fonctions :

```

utop # test_equisat_quine' ["a"; "b"; "c"; "d"] 5;;
F = (c + ((d * c) * (a + (c * a))))
T(F) = (new_0 * (((new_0 <-> (old_c + new_1)) * (((new_1 <-> (new_2 * new_3))
      * (new_2 <-> (old_d * old_c))) * ((new_3 <-> (old_a + new_4))
      * (new_4 <-> (old_c * old_a)))))))
quine_sat(F) : true
quine_sat(T(F)) : true
Equisat ? true
- : unit = ()

utop # test_equisat_dp11' ["a"; "b"; "c"; "d"] 5;;
F = ((d * (b -> (b -> c))) + (c + c))
T(F) = (new_0 * (((new_0 <-> (new_1 + new_4)) * ((new_1 <-> (old_d * new_2))
      * ((new_2 <-> (old_b -> new_3)) * (new_3 <-> (old_b -> old_c))))))
      * (new_4 <-> (old_c + old_c))))
is_ext_sat T(F) ? true
is'_rest_sat F ? true
- : unit = ()

```

7

Évaluation des performances

Dans le fichier `Benchmark.ml`, donner corps aux fonctions suivantes :

- `bench_dp11` prenant en paramètre une formule `f` (sans occurrence de l'opérateur d'équivalence) et qui réalise la séquence suivante :
 1. affiche `f` au format infixe,
 2. calcule la transformée `f'` de `f` en mesurant son temps de calcul, affichant ces deux informations,
 3. calcule la FNC `fcc` associée à `f` en mesurant son temps de calcul, affichant ce temps de calcul, le nombre de clauses et de littéraux,
 4. calcule la FNC `fcc` associée à `f'` en mesurant son temps de calcul, affichant ce temps de calcul, le nombre de clauses et de littéraux,
 5. détermine la satisfaisabilité de `fcc` avec `dp11_sat_unit_prop` en mesurant son temps de calcul, affichant ces deux informations,
 6. détermine la satisfaisabilité de `fcc'` avec `dp11_sat_unit_prop` en mesurant son temps de calcul, affichant ces deux informations.
- `bench_dp11'` prenant en paramètres un alphabet et un nombre d'opérateurs, calculant une formule aléatoire en utilisant ces deux paramètres, puis appelant `bench_dp11` sur cette formule aléatoire.

Voici un exemple de sortie lors de l'exécution de cette dernière fonction :

```

utop # bench_dp11' ["a"; "b"; "c"; "d"] 5;;
F = (((b -> (b -> d)) -> ⊥) + d)
T(F) = (new_0 * (((new_0 <-> (new_1 + old_d)) * ((new_1 <-> (new_2 -> ⊥))
      * ((new_2 <-> (old_b -> new_3)) * (new_3 <-> (old_b -> old_d))))))
Transformation : 0.000010s
fcc (F) : 0.000051s (2 clauses, 4 littéraux)
fcc (T(F)) : 0.000036s (12 clauses, 26 littéraux)
Dp11 (F) : true (0.000003s)
Dp11 (T(F)) : true (0.000044s)
- : unit = ()

```

Pour mesurer le temps d'exécution d'une fonction, vous pourrez utiliser la fonction suivante :

```
(** Mesure du temps d'exécution d'une formule fonc sur un paramètre arg
renvoyant le couple (résultat, temps de calcul). *)
let mesure_fonc arg =
  let debut = Sys.time () in
  let res = fonc arg in
  let fin = Sys.time () in
  (res, fin -. debut)
```

8 Structure des ressources de départ

La structure du projet et l'utilisation de dune sont imposées. Les formules utilisent les opérateurs classiques, mais également l'opérateur "équivalence" $\leftrightarrow$.

Le projet est constitué d'un dossier Proposition contenant huit fichiers ml. Vous devrez copier-coller certaines fonctions réalisées en TP dans les fichiers suivants :

- Formule.ml;
- Quine.ml;
- FCC.ml;
- DPLL.ml.

En utilisant les fichiers précédents, vous devrez compléter les fichiers suivants :

- le fichier RandomFormule.ml contenant une fonction de génération aléatoire de formules;
- le fichier Tseitin.ml contenant les fonctions concernant la transformation de Tseitin;
- le fichier CorrecTest.ml contenant les tests de correction;
- le fichier Benchmark.ml contenant les mesures de performances.

9 Barème indicatif

1. Code (14 points) :
 - (a) Module RandomFormule (2 points) :
 - génération aléatoire d'une formule (2 points)
 - (b) Module Tseitin (6 points) :
 - transformée (3 points)
 - conversions d'interprétation (3 points)
 - (c) Module CorrecTest (3 points) :
 - fonction test_equisat_quine (1 point)
 - fonction test_equisat_dp11 (2 points)
 - (d) Module Benchmark : fonction bench_dp11 (3 points)
2. Rapport et soutenance (6 points)

Il s'agit d'un barème indicatif "de départ". À celui-ci s'ajouteront différentes pénalités, **notamment** si :

- votre code rencontre des erreurs de compilation;
- votre code émet des avertissements à la compilation;
- des contre-exemples sont détectés,

ainsi que des bonus, **notamment** si :

- votre rapport est écrit en $\text{\LaTeX}$,
- vous codez d'une façon propre, lisible, élégante et efficace.

Le module Random permet la génération (pseudo-) aléatoire de différents types de base. L'expression `Random.int b` permet de générer un nombre entre 0 (compris) et b (exclus), où b est un entier. Certaines fonctions utilisent le type `unit`, type ne possédant qu'une seule valeur, la valeur `()`. Cela est fait pour distinguer les constantes des fonctions : la génération aléatoire se doit d'être réalisée par des fonctions. Par exemple, comparez les éléments suivants :

```
let x = Random.int 30;;
let y () = Random.int 30;;
x;;
x;;
y ();;
y ();;
```

Ainsi, pour réaliser une fonction de génération d'un couple de caractères, comparez

```
let random_chars =
  (char_of_int (Random.int 256), char_of_int (Random.int 256));;
let random_chars' () =
  (char_of_int (Random.int 256), char_of_int (Random.int 256));;
random_chars;;
random_chars;;
random_chars' ();;
random_chars' ();;
```

Enfin, pour correctement utiliser les générateurs, il faut initialiser le germe en utilisant `Random.self_init ()`, initialisant celui-ci d'une façon (pseudo-) aléatoire.